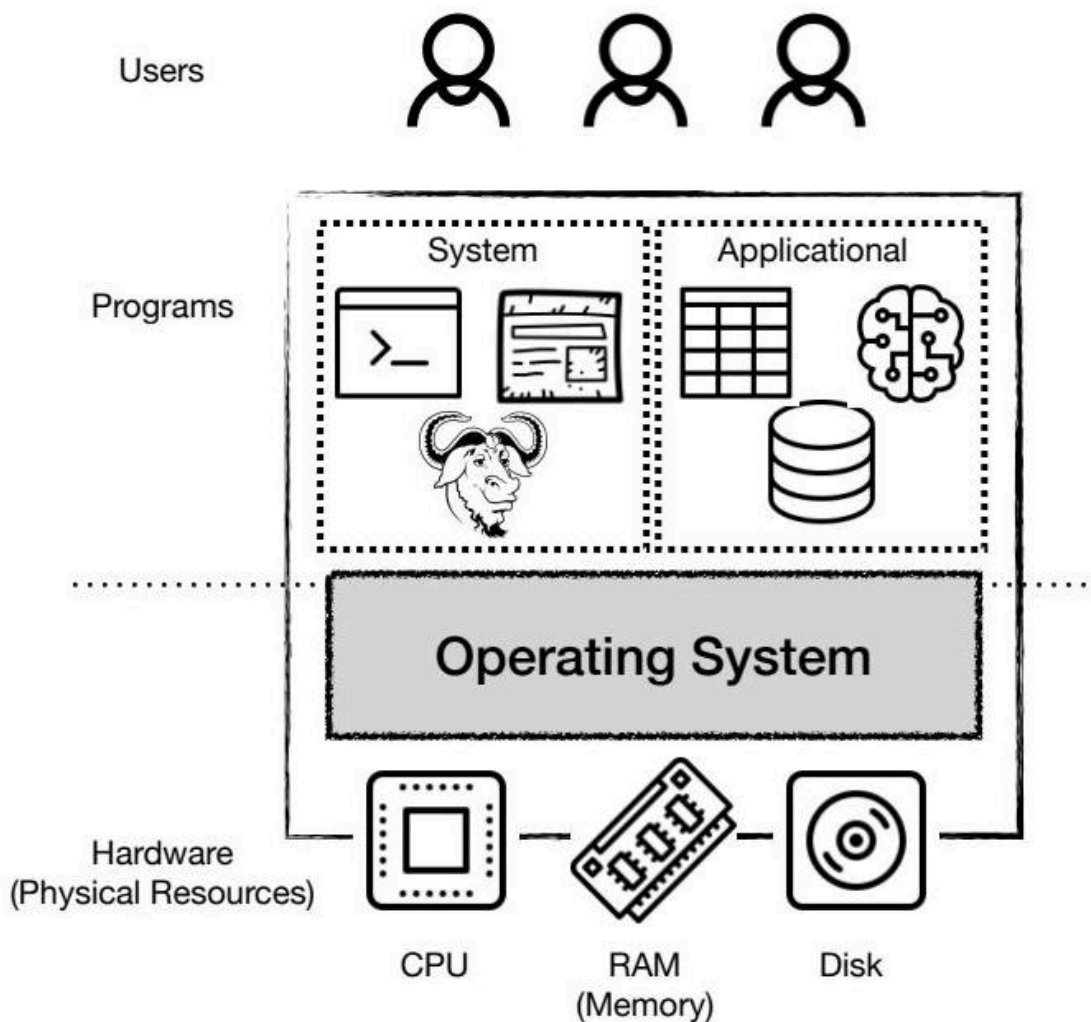


Sistemas Computacionais Modernos

Um sistema computacional moderno é composto por:

- **Usuários:** Humanos ou outros computadores (ex: conexão remota).
- **Programas Aplicacionais:** Programas genéricos como editores de texto, planilhas, bancos de dados, engines de IA, jogos, etc.
- **Programas de Sistema:** Programas especializados como shell (ex: bash), interfaces gráficas (GUI), compiladores, bibliotecas.
- **Sistema Operacional (SO):** Gerencia a interação entre usuários/programas e o hardware.
- **Hardware:** CPU, memória, dispositivos de I/O (ex: disco, teclado, display).

Aqui está um diagrama ilustrando os componentes de um sistema de computador, mostrando as relações entre usuários, programas e hardware. Ele oferece uma visão geral abrangente de como esses elementos interagem dentro de um sistema computacional.



Programas do Sistema Operacional

- Os principais clientes do SO são **programas**.
- Um programa em execução executa instruções (bilhões por segundo em programas modernos).

O que acontece quando um programa é executado?

1. O processador (CPU) busca uma instrução da memória. A instrução é decodificada e executada (ex: somar números, acessar memória, verificar condições).
2. Este ciclo é repetido até que todas as instruções sejam concluídas, ou seja, o programa termina.
3. A CPU tem um pequeno número de registradores para guardar informações temporárias (ex: Program Counter (PC), Instruction Register (IR)).

Propósito do Sistema Operacional

O principal objetivo do SO é tornar **fácil** e **eficiente** a execução de programas.

- Permitir que múltiplos programas sejam executados simultaneamente (com recursos de hardware limitados).
- Permitir que programas compartilhem memória e acessem dispositivos de I/O (ex: disco).
- Gerenciar eficientemente os recursos físicos utilizados pelos programas.

Evolução dos Sistemas Operacionais

No Início (1954)

- Computadores (sistemas mainframe) eram operados manualmente por usuários.
- Existia um conjunto comum de bibliotecas (ex: para abstrair código de I/O de baixo nível).
- Geralmente, um programa era executado por vez, com cada usuário tendo um slot de tempo atribuído.
- Altamente ineficiente e caro.

"Cada usuário recebia um slot mínimo de 15 minutos, dos quais geralmente gastava 10 minutos configurando o equipamento para fazer sua computação." - George Ryckman sobre o computador IBM 701 open shop

O custo do tempo de computador desperdiçado era de \$146.000 por mês, em 1954!

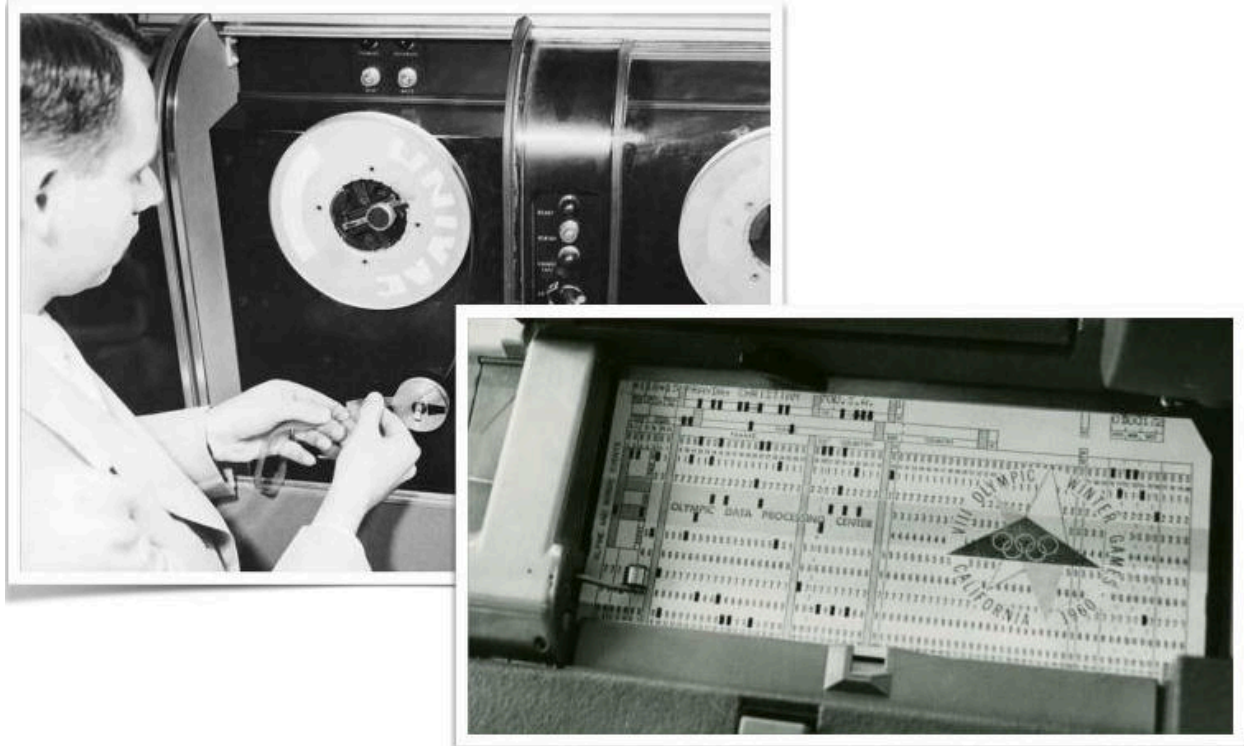
Esta imagem retrata um sistema de computador vintage de grande porte, com várias unidades interconectadas e um painel de controle visível, refletindo a tecnologia da computação inicial.

 Computador antigo

O Operador (1959)

- Usuários foram removidos da operação do computador.
- Usuários preparavam seus programas em cartões perfurados para serem entregues a um operador.
- O operador (ainda um humano) carregava os programas em fitas magnéticas.
- Programas executados sequencialmente (como armazenados na fita) - processamento em lote.
- Mais eficiente, mas menos conveniente para os usuários.
- Dependente da ordem de agendamento decidida pelo operador.
- Não é possível interagir com o programa em execução (ex: para depuração).

Esta imagem composta mostra um operador trabalhando em um computador UNIVAC, destacando o papel do operador na computação inicial e um close-up de um painel de controle que mostra a interface para gerenciar essas máquinas.



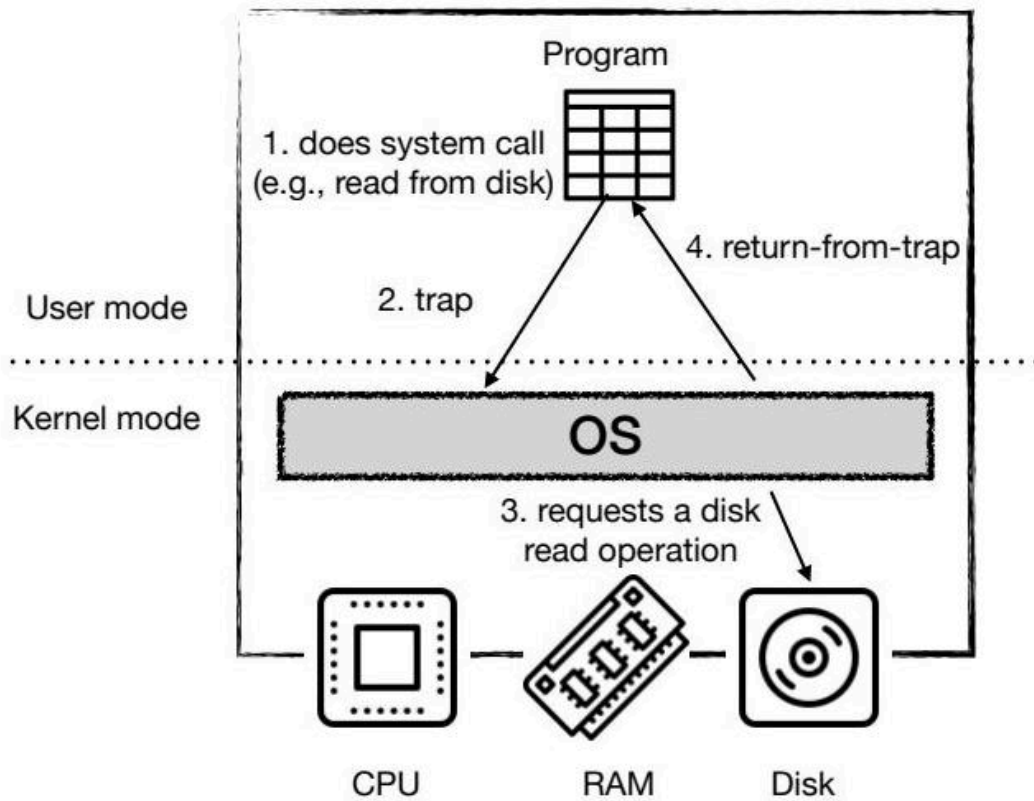
Programa de Controle (Anos 1960)

- E se as tarefas feitas pelo operador pudessem ser automatizadas?
- Um programa de controle especial ("SO") poderia gerenciar os recursos do computador, agendar programas de usuário (jobs) e executá-los.
- Usuários (ou o operador) apenas carregariam/descarregariam programas (gerenciar I/O).
- O que acontece se o programa de controle perder o controle?
 - Um job entra em um loop infinito e assume o controle do processador.
 - Um job grava erroneamente na região de memória do programa de controle.
 - Um job leva a erros ao acessar dispositivos de I/O.
- Os desafios anteriores são apenas solucionáveis com a evolução do hardware.
 - **Interrupções de hardware:** Programas de usuário podem ser interrompidos, mesmo que estejam em um loop infinito, devolvendo o controle ao programa de controle.
 - **Níveis de privilégio de hardware:** Programas são executados em modo de usuário (sem privilégios). O programa de controle (SO) é executado em modo kernel (com privilégios). Operações privilegiadas (ex: memória, disco, operações de rede) só são feitas em modo kernel.
 - **Proteção de memória:** Evita que programas acessem erroneamente a memória do programa de controle e a destruam.

Chamada de Sistema (Anos 1960)

- A ideia de **chamada de sistema** nasce com o sistema de computação Atlas!
- Quando programas, executados em modo de usuário, devem fazer operações privilegiadas (ex: acessar disco, alocar memória), eles emitem chamadas de sistema.
- Quando uma chamada de sistema é iniciada (instrução de hardware trap), o hardware eleva o nível de privilégio para o modo kernel.
- No modo kernel, o SO faz as operações privilegiadas.
- Após atender à solicitação, o SO chama uma instrução de retorno de trap, que reverte para o modo de usuário e retorna o controle para onde o programa parou.

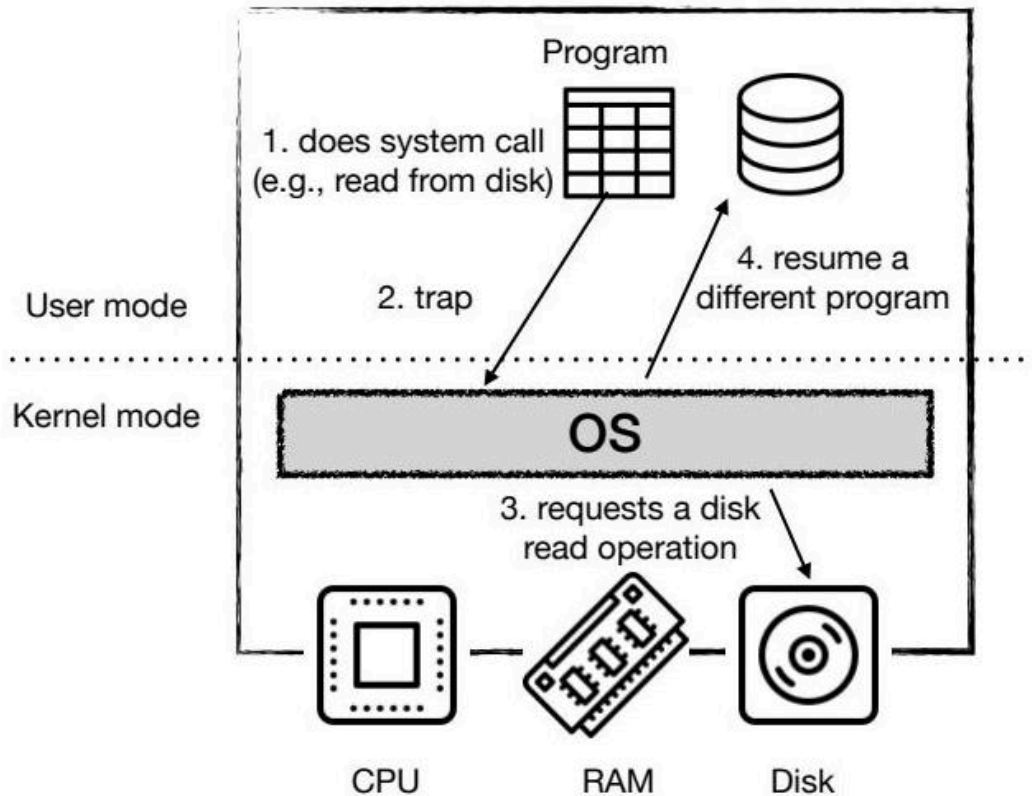
Este fluxograma ilustra o processo de uma chamada de sistema para uma operação de leitura de disco, demonstrando a transição entre o modo de usuário e o modo kernel e o papel do SO na manipulação de solicitações de hardware.



Multiprogramação e Time-sharing (Anos 1960/70)

- Com a evolução do hardware, o SO agora pode carregar múltiplos programas na memória e alternar rapidamente entre eles.
- No entanto, operações de I/O são lentas... Programas esperando na CPU para que as solicitações de I/O sejam concluídas estão desperdiçando ciclos de CPU.
- Alternar programas esperando por I/O - multiprogramação!
 - Após lidar com uma chamada de sistema (ex: instruir o disco a ler dados), o SO alterna para outro programa (ou seja, em vez de esperar que os dados sejam lidos).
 - Quando os dados são lidos do disco e colocados em um buffer de memória, o SO é notificado com a ajuda de uma interrupção de hardware (e pode então retomar o programa que emitiu a chamada de sistema).
- O minicomputador emerge, é mais acessível e utilizável por um grupo menor de usuários.
- O tempo da CPU agora é compartilhado entre os jobs de vários usuários (com a ajuda do SO). UNIX nasce!
- Na multiprogramação, um programa mantém a CPU até que I/O seja necessário. E se o programa nunca fizer I/O?
- **Time-sharing** (ou multitasking) é uma extensão lógica da multiprogramação em que os programas são frequentemente alternados (com a ajuda de uma interrupção de hardware do timer).
- Os usuários têm a ilusão de ter a máquina só para si (tempos de resposta mais baixos).
- Novos problemas interessantes surgem:
 - Agendamento eficiente de processos.
 - Isolamento de memória entre processos em execução.
 - Concorrência.

Esta imagem mostra um fluxograma ilustrando o processo de um programa fazendo uma chamada de sistema para ler de um disco, demonstrando como o SO gerencia a operação entre o modo de usuário e o modo kernel.



Computadores Pessoais (Anos 1970)

- Custos mais baixos agora permitem que cada usuário tenha seu próprio computador (mais conveniente!).
- Infelizmente, os SOs deram um passo atrás em eficiência e proteção.
 - O DOS tinha vários problemas de proteção de memória.
 - As primeiras versões do MAC OS congelavam devido a programas com loops infinitos.
- Mais tarde, as ideias interessantes dos minicomputadores (versões adaptadas do UNIX) começaram a ser portadas para PCs.
 - LINUX nasce!
- E então veio a web, sistemas distribuídos, smartphones, dispositivos IoT...

A imagem mostra um sistema de computador inicial dos anos 1960 ou 1970, ao lado de um moderno computador de placa única.



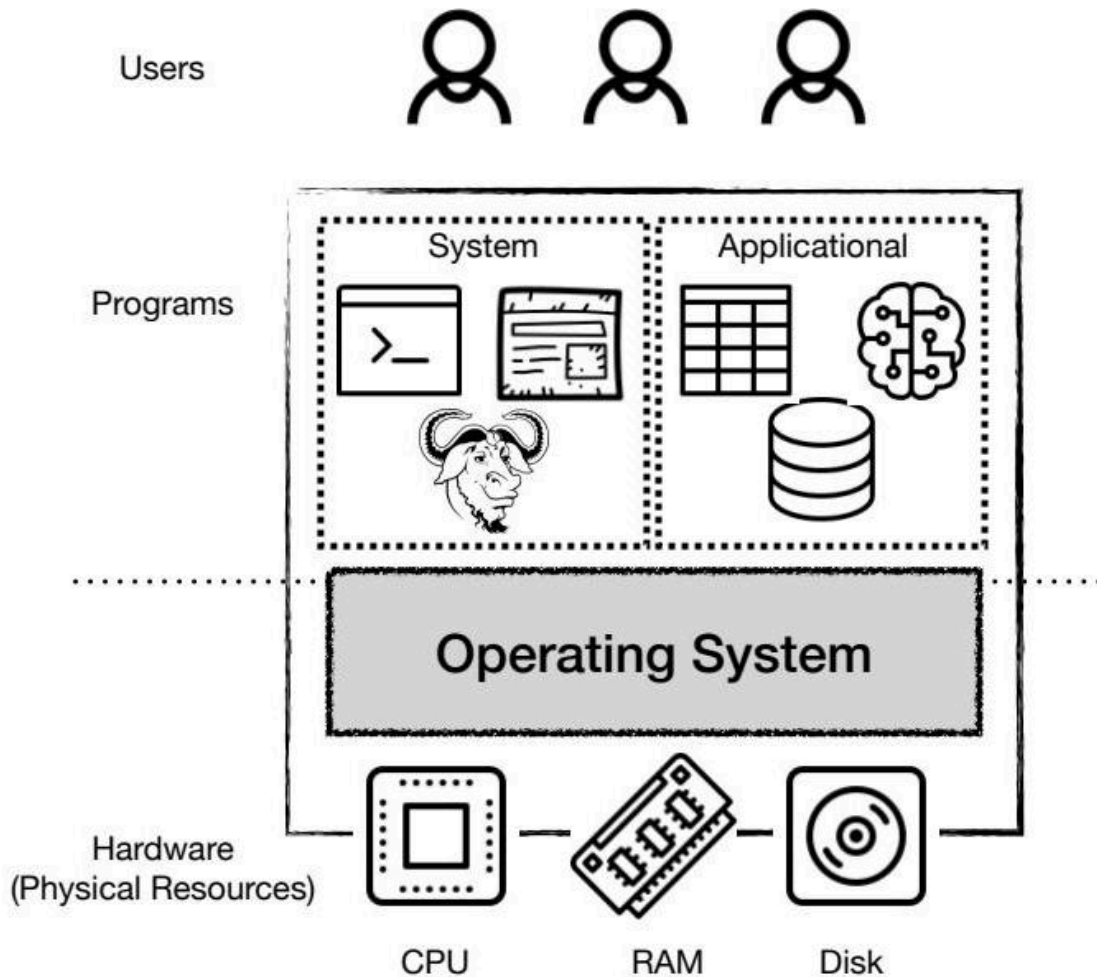
A imagem retrata a comparação entre um computador pessoal vintage dos anos 70 e um moderno Raspberry Pi.



Sistemas Operacionais Modernos

Voltando da nossa "não tão pequena" digressão histórica...

A imagem ilustra os componentes de um sistema computacional moderno, mostrando as relações entre usuários, programas e hardware.



Princípios de Design

- Abstrair os detalhes de "baixo nível" do hardware.
 - Virtualiza recursos físicos, abstraindo sua complexidade e heterogeneidade (ex: arquiteturas de hardware, modelos de disco, modelos de GPU).
 - Ex: Linux roda sob múltiplas arquiteturas de hardware.
- Fornecer uma interface (API de Chamada de Sistema) para que os usuários digam ao SO o que fazer.
 - Para executar programas: `fork()`, `exec()`, ...
 - Gerenciar memória: `brk()`, ...
 - Acessar dispositivos: `open()`, `read()`, `write()`, ...

Esses princípios de design fornecem **Conveniência** aos usuários!

Chamadas de Sistema

- Em cima do SO, pode-se construir vários programas de sistema interessantes:
 - Interfaces gráficas de usuário (GUI).
 - Interfaces de linha de comando (shell).
 - Ferramentas para compilar, carregar, executar e depurar programas.
 - Ferramentas para manipular e explorar arquivos, diretórios.
- A maioria dos usuários vê o SO como a combinação de seu kernel (código central) e programas de sistema, não percebendo a existência da API de chamada de sistema. (É discutível se os programas de sistema podem ser considerados parte do SO).
- Felizmente, neste curso, examinaremos os conceitos principais do kernel do SO e trabalharemos com a API de chamada de sistema!
 - Crucial para qualquer pessoa que deseje construir aplicações melhores e entender seus sistemas.

Princípios de Design

- Garantir o desempenho ao executar múltiplos programas acessando recursos físicos compartilhados.
 - Qual programa deve ser agendado em seguida? Qual programa deve gravar dados no disco em seguida?
 - Importante: o SO deve tomar decisões rapidamente e sem sobrecarga excessiva de uso de recursos.
- Garantir a proteção, isolando os programas em execução uns dos outros e do SO.
 - Um programa malicioso/bugado não pode acessar regiões de memória privadas de outros programas (e SO).
 - Além disso, não deve ser capaz de dominar a CPU e "bloquear" o computador.
- Outros princípios:
 - **Confiabilidade**: Uma falha do SO significa que todas as aplicações falharão.
 - **Segurança**: Uma extensão do princípio de proteção contra programas maliciosos (ex: ransomware).
 - Eficiência energética, mobilidade, ...

Gerenciamento de CPU

- Embora um computador tenha um número limitado de CPUs, os usuários têm a impressão de que vários programas (muito mais do que as CPUs disponíveis) estão sendo executados simultaneamente.
 - Como o SO fornece essa ilusão?
 - Como os usuários podem usar as APIs do SO para iniciar e parar programas?
 - Como o SO escolhe quais programas estarão realmente sendo executados a cada momento?
- Estudaremos abstrações, mecanismos e políticas empregadas pelo SO.
- A separação entre **mecanismo** e **política** é um paradigma de design comum usado pelo SO.
 - Mecanismo: como o SO alterna os programas em execução?
 - Política: qual programa deve ser agendado em seguida?

Gerenciamento de Memória

- A memória física pode ser vista como um simples array de bytes. Programas (em execução) precisam armazenar suas instruções e estruturas de dados na memória.
 - Como o SO abstrai e gerencia os acessos à memória feitos pelos programas?
 - Como é possível ter múltiplos programas compartilhando com segurança a mesma memória física?
 - Como o SO fornece a ilusão de ter mais memória do que a fisicamente disponível para executar programas?
- Estudaremos estruturas, mecanismos e políticas do SO que respondem a essas perguntas.

Persistência

- A memória é volátil, o que significa que os dados são perdidos após uma reinicialização (ou falha) do computador. Dispositivos de disco (ex: HDDs, SSDs) permitem que os programas armazenem seus dados de forma persistente.
 - Como o SO abstrai o acesso (API) aos drivers de dispositivo de baixo nível desses discos?
 - Como o SO torna as operações de disco eficientes quando múltiplos programas usam o mesmo dispositivo simultaneamente?
 - Normalmente acessamos nossos dados usando uma abstração baseada em arquivos (arquivos, pastas, ...). Como o SO habilita tal abstração?
- Estudaremos as propriedades e o design de dispositivos de disco e sistemas de arquivos para responder a essas perguntas.